

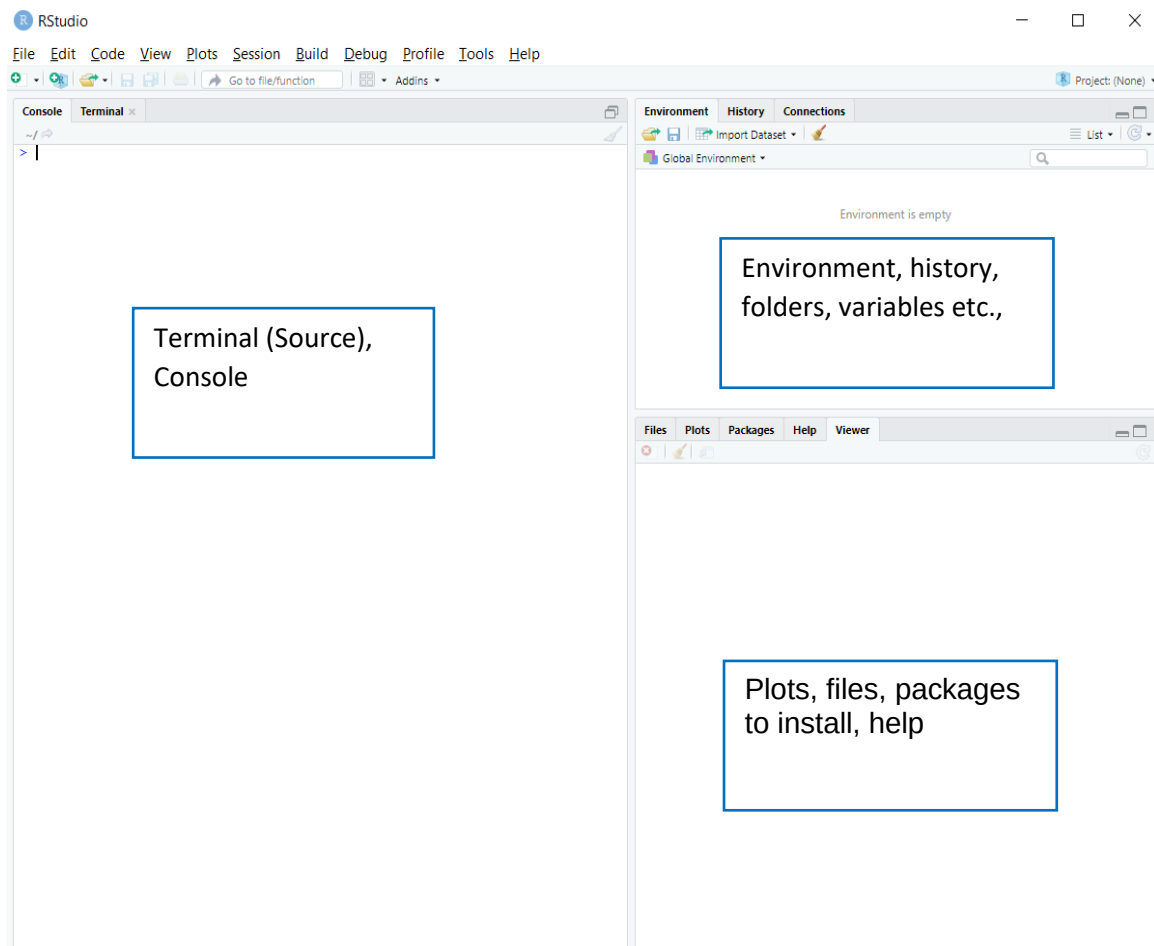
Lab 01 – R, RStudio Setup and Basic Learning

RStudio is an integrated development environment for R, with a console, syntax-highlighting editor that supports direct code execution, and tools for plotting, history, debugging and workspace management.

TASK 1: Familiarize with R/RStudio

1. RStudio Environment

Open RStudio from the Windows Program-list. You should see as below



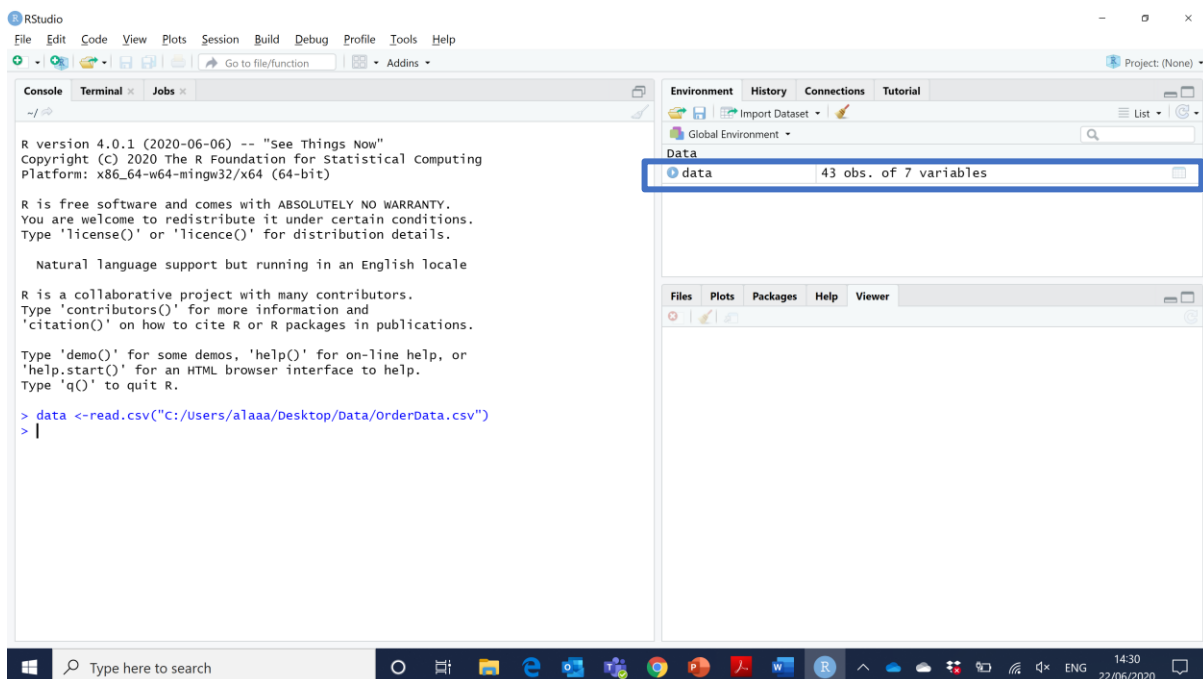
2. Importing Data

Download the OrderData.csv and OrderData.xlsx files from the Blackboard Week 1 Learning Material and place into your personal Network drive. To import data from a .CSV file, use the following command:

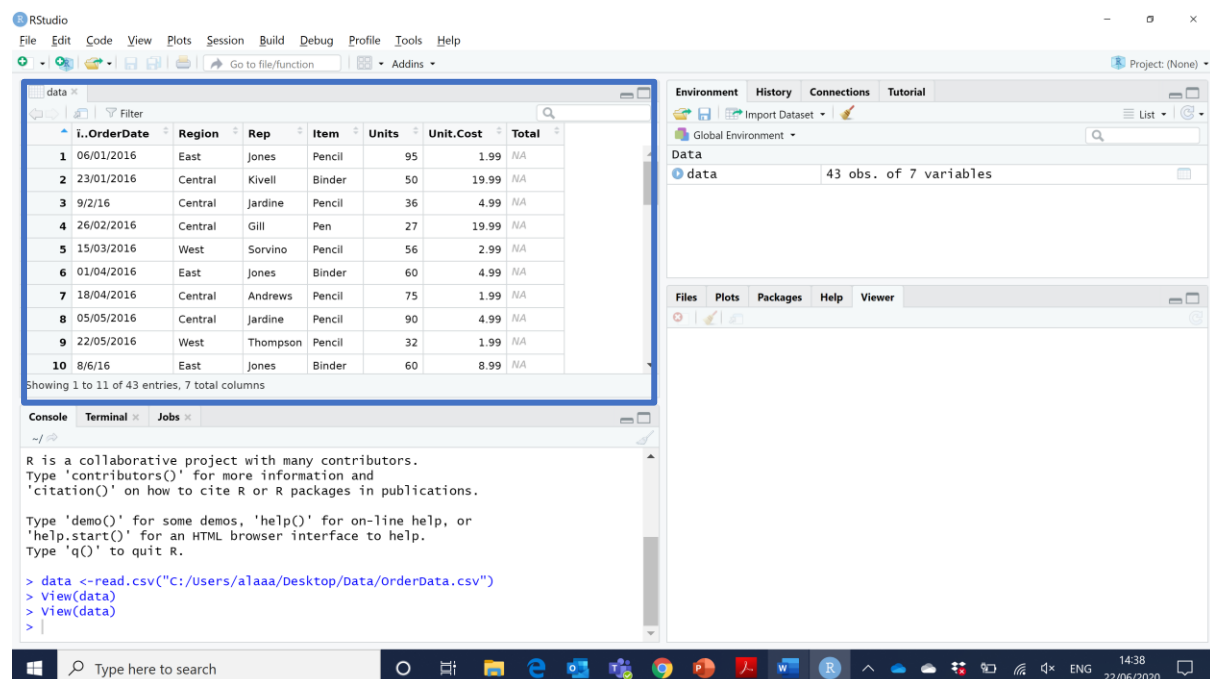
```
data <-read.csv("Data file path.csv")
```

e.g. `data <- read.csv("C:/data/R/OrderData.csv")`

You can copy and paste the path of your .csv file into above command. Your environment should look like:



If you click on the >data tab in top right Global Environment window, you will be able to view the data within the Source window on Top-Left window as below.



Try the following very useful commands to view the variable 'data'

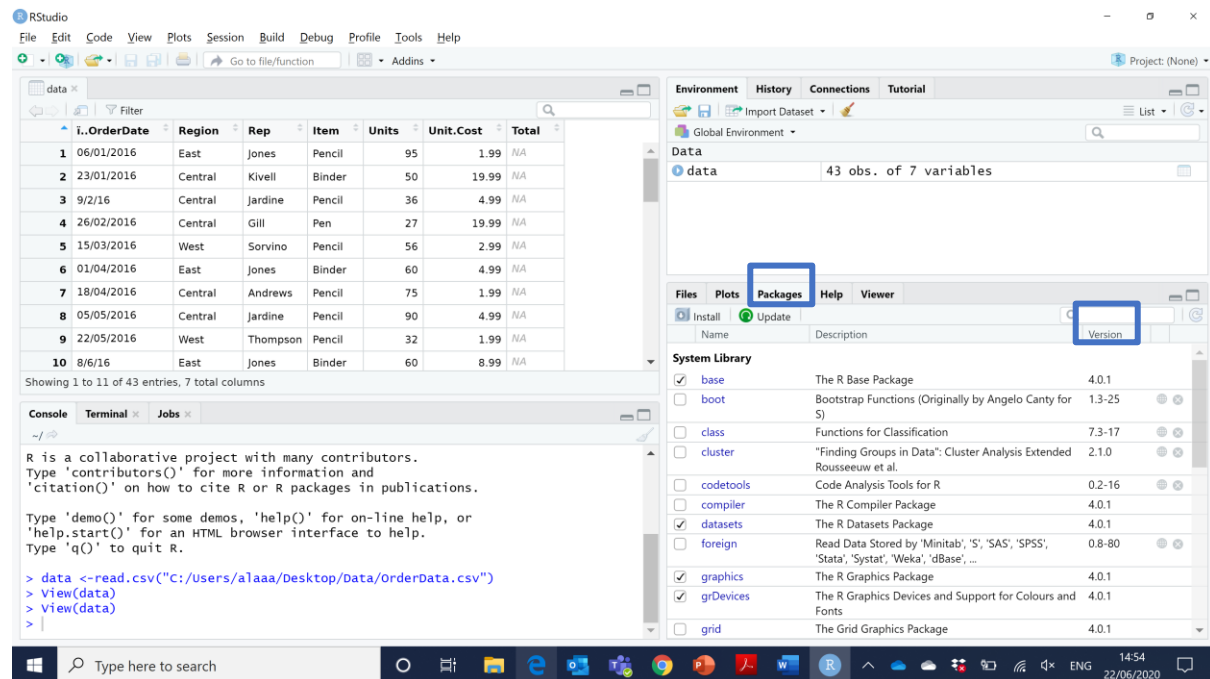
print(data)	prints the entire object (avoid with large tables)
head(data)	prints the first 6 lines of your data
str(data)	shows the data structure of an R object
names(data)	lists the column names (i.e., headers) of your data
ls(data)	lists all the R objects in your workspace directory

3. R Packages & Installations

Packages are collections of additional functions that can be loaded on demand. They commonly include example data that can be used to demonstrate those functions. Although R comes with many common statistical functions and models, most of our work requires additional packages.

To use a package, you must first install it and then load it. These steps can be done at the command line or using the Packages Tab. Every time you start a new R session, however, you need to load every package that you intend to use in that session.

Using the 'Packages' tab in Bottom-Right plot/resources window, you can find required Package in the Search area (text box) as highlighted in the figure below. Then select the right package you needed, and press 'Install' tab below the 'Packages' as shown in figure below.

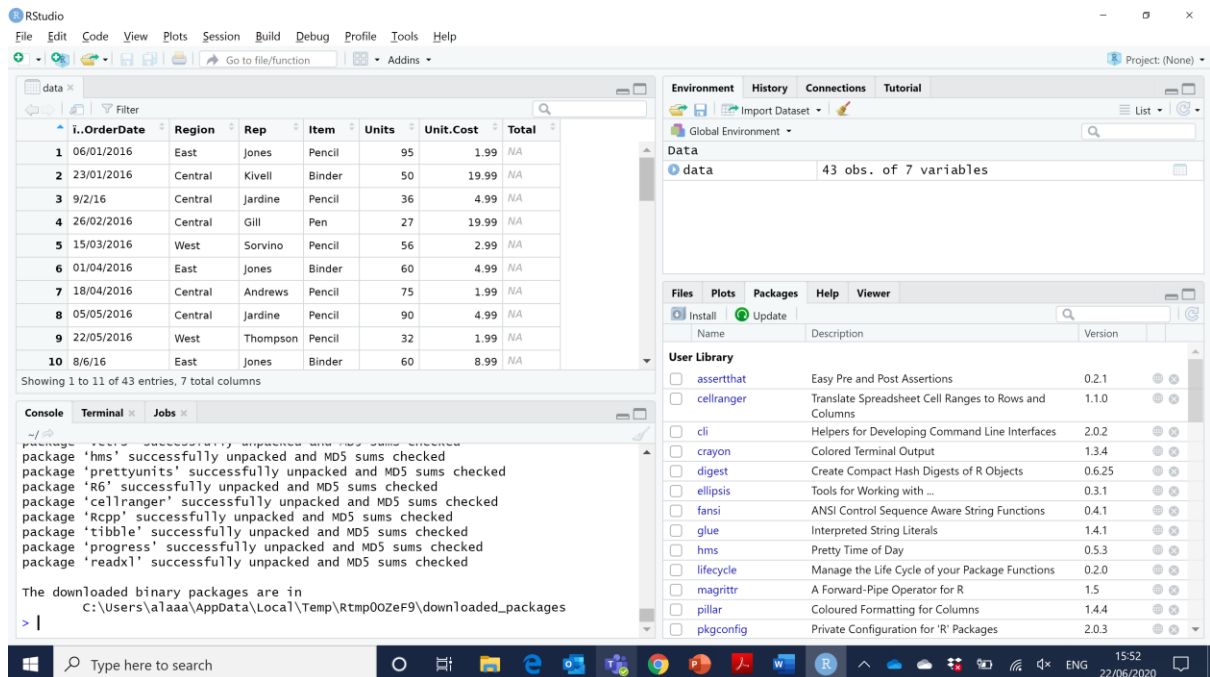


Example: Reading from Excel files (Source: ncss-tech.github.io)

R can import Excel files, but it is a bad idea to use Excel. Excel has a dangerous default which automatically converts data with common notations to their standard format without warning or notice. For example, the character "11-JUN" entered into a cell automatically becomes the date 6/11/2017, even though the data is still displayed as 11-JUN. The only way to avoid this default behavior is to manually import your data into Excel via the **Data Tab>Get External Data Ribbon**, and manually set the data type of all your columns to text. Failure to do so has resulted in numerous retracted research articles (Washington Post Article).

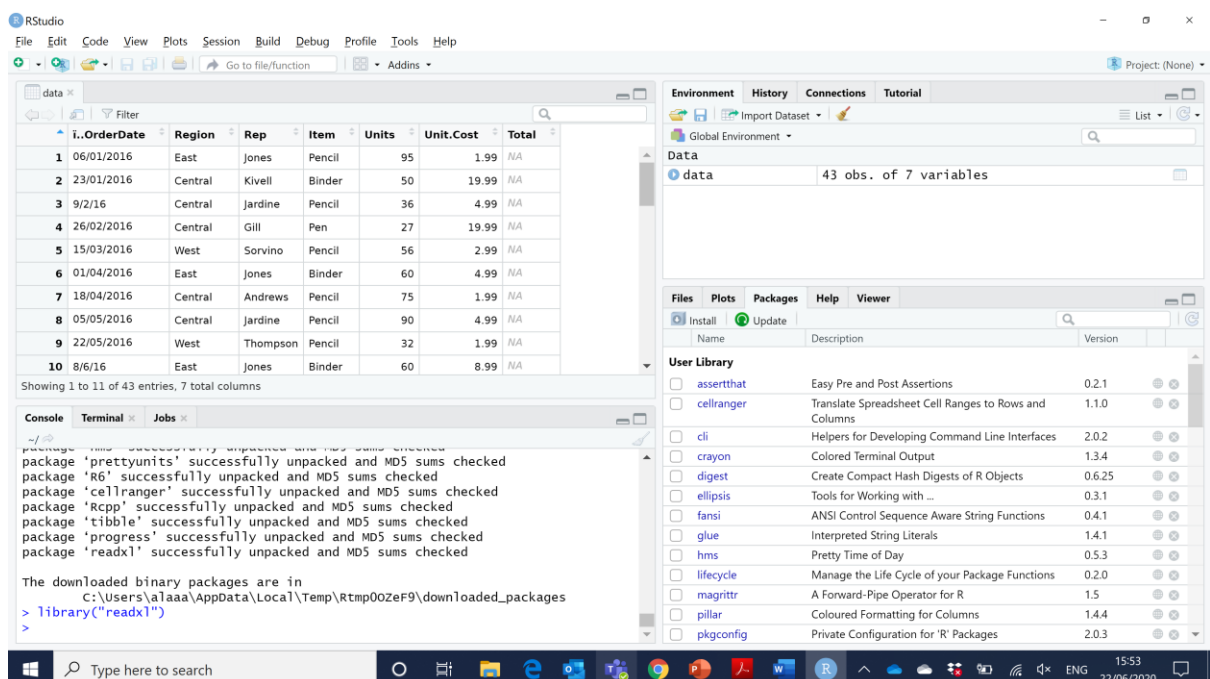
The **readxl** package, developed by Hadley Wickham, can be used to easily import Excel files (xls|xlsx) into R without any external dependencies. To install the required package, write the following command:

install.packages("readxl")



Then Load the installed Library using:

library("readxl")



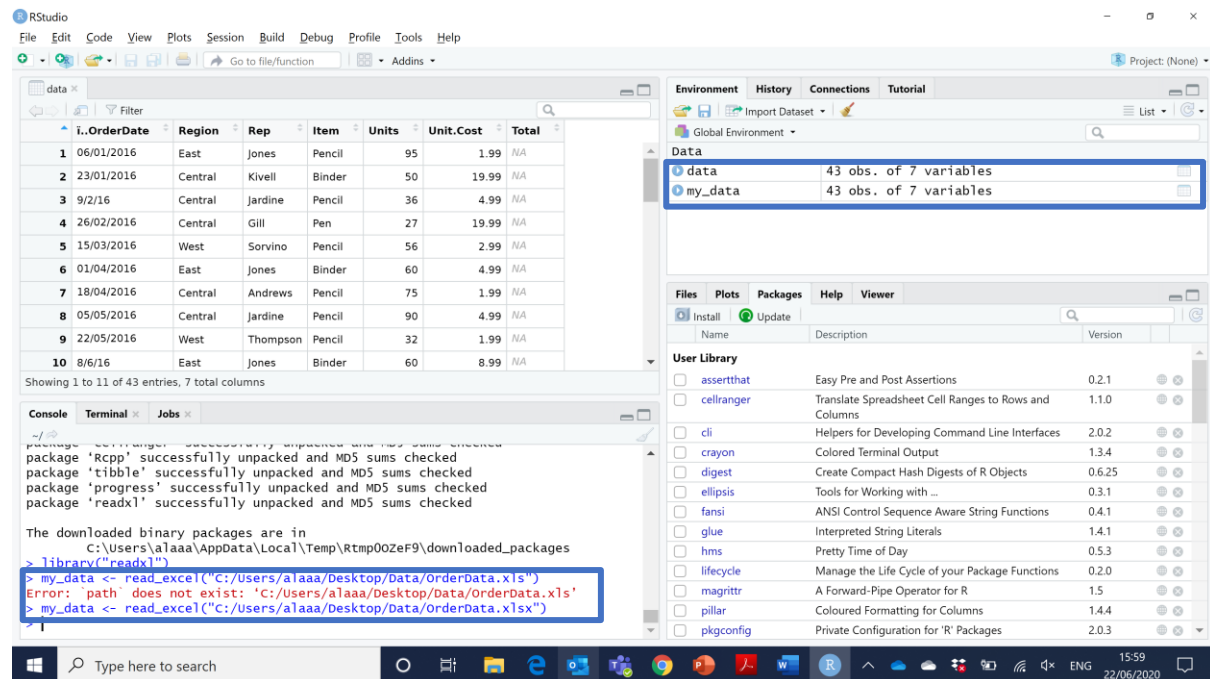
Now, you can read the excel data as below

```
my_data <- read_excel("path of your excel file.xls")
```

```
my_data <- read_excel("path of excel file.xlsx")
```

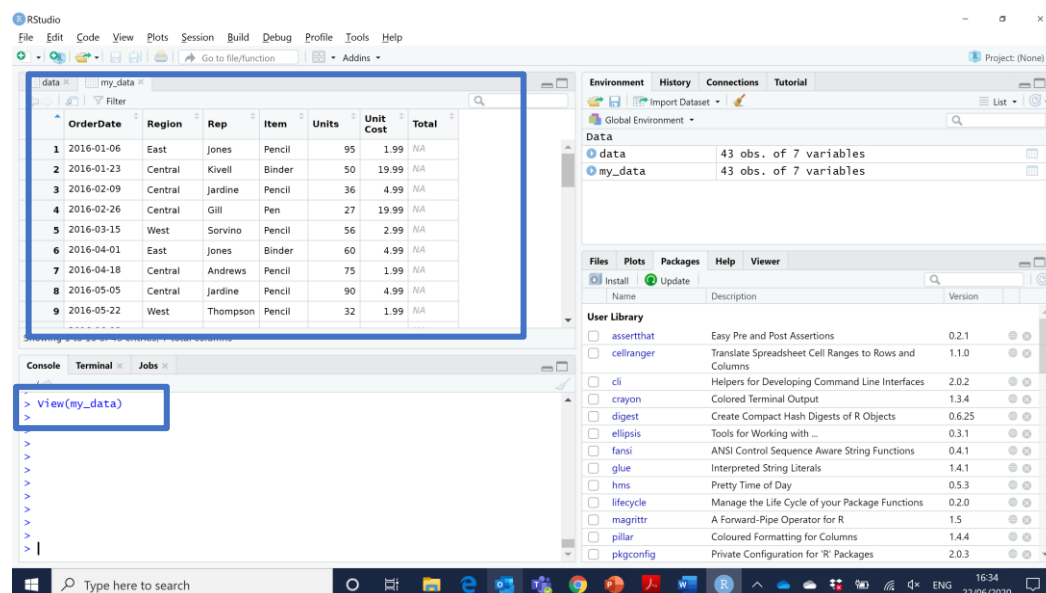
You will read the xls or xlsx file and name it **my_data**

After running the above commands, your Environment should look like:



To view the excel file use the following command

```
View(my_data)
```

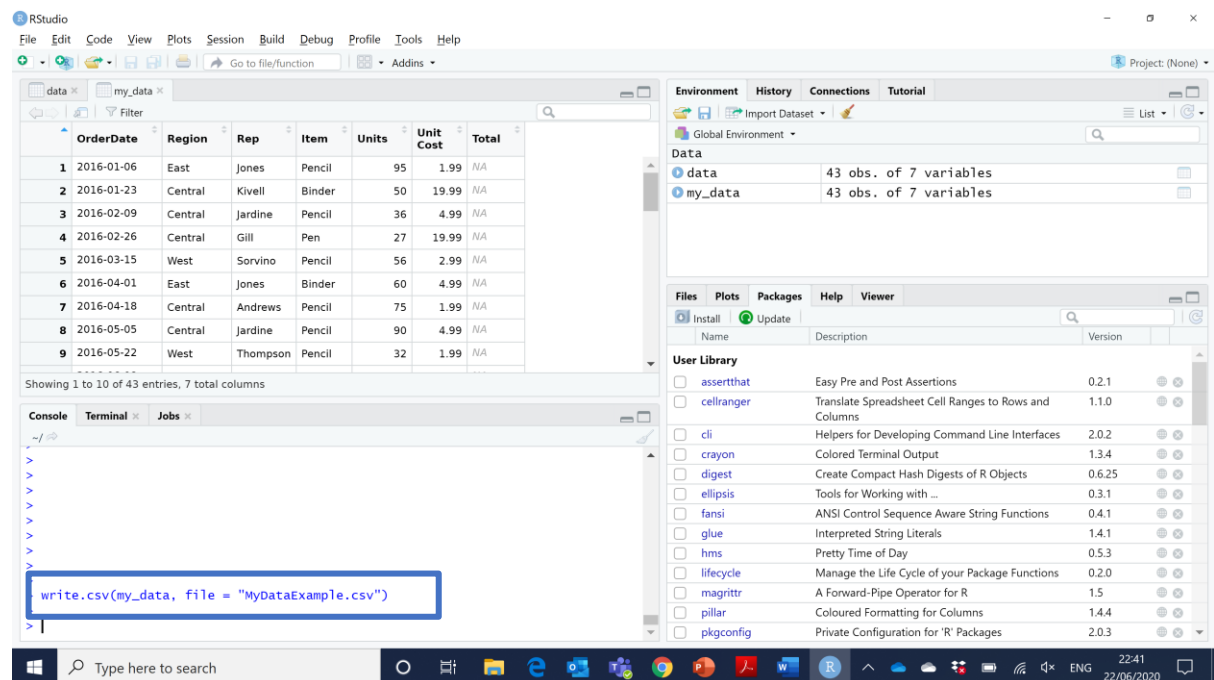


4. Exporting Data

To export data from R, use the command `write.csv()` function. Since we have already set our working directory, R automatically saves our file into the working directory.

```
write.csv(my_data, file = "MyDataExample.csv")
```

In this command the file `my_data` will be saved under the name “MyDataExample.csv” in Documents as it is my working directory.



or use the `write.table()` function to export other text file types

5. Basic Tips of Scripting

To write a script, simply open a new R script file by clicking **File>New File>R Script**. Within the text editor **type** out a sequence of functions.

- Place each function (e.g. `read.csv()`) on a separate line.
- If a function has a long list of arguments, place each argument on a separate line.
- A command can be executed from the text editor by placing the cursor on a line and typing **Ctrl + Enter**, or by **clicking** the Run button.
- An entire R script file can be executed by **clicking** the Source button.

6. Saving R Files

In R, you can save several types of files to keep track of the work you do. The file types include: workspace, script, history, and graphics. It is important to save often because R, like any other software, may crash periodically. Such problems are especially likely when working with large files. You can save your workspace in R via the command line or the File menu.

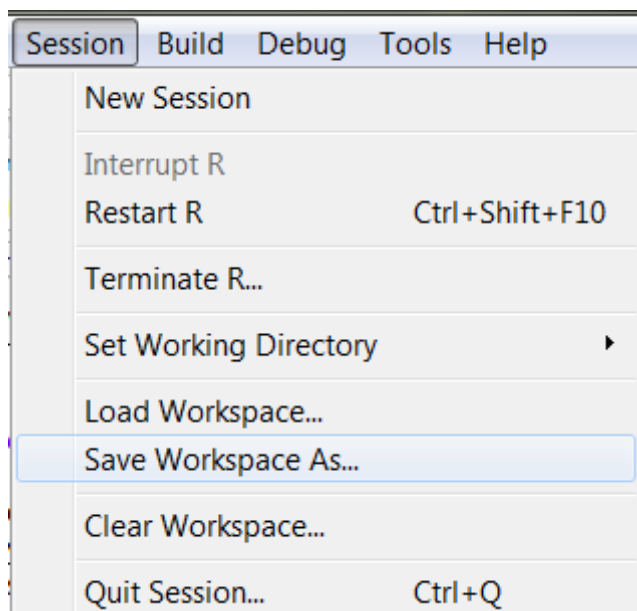
a) R script (.R)

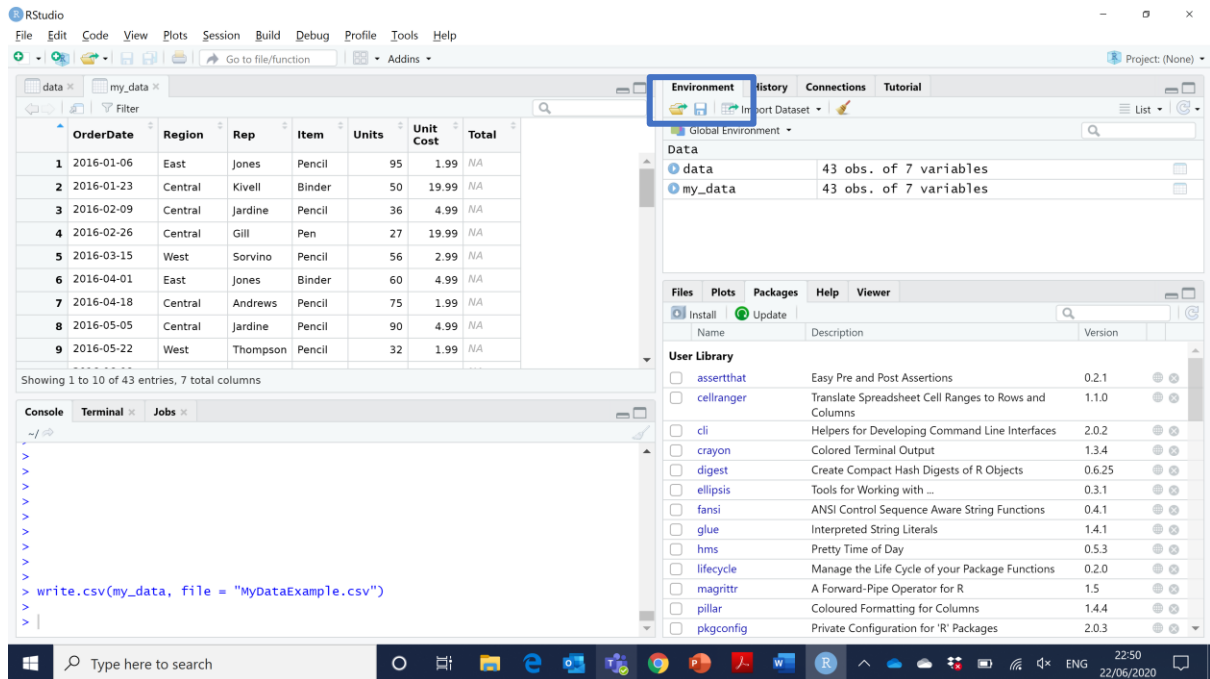
An R script is simply a text file of R commands that you've typed.

You may want to save your scripts (whether they were written in R Editor or another program such as Notepad) so that you can reference them in the future, edit them as needed, and keep track of what you've done. To save R scripts in RStudio, simply **click the save button** from your R script tab. Save scripts with the .R extension. R assumes that script files are saved with only that extension. If you are using another text editor, you won't need to worry about saving your scripts in R.

b) Workspace (.Rdata)

The R workspace consists of all the data objects you've created or loaded during your R session. When you quit R by either typing `q()` or exiting out of the application window, R will prompt you to save your workspace. If you choose yes, R saves a file named .RData to your working directory. The next time you open R and reload your .Rdata workspace, all of your data objects will be available in R and all of the commands that you've typed will be accessible by using the up-arrow and down-arrow keys on your keyboard. You can also save or load your workspace at any time during your R session from the menu by clicking **Session>Save Workspace As...**, or the save button on the **Environment Tab**.



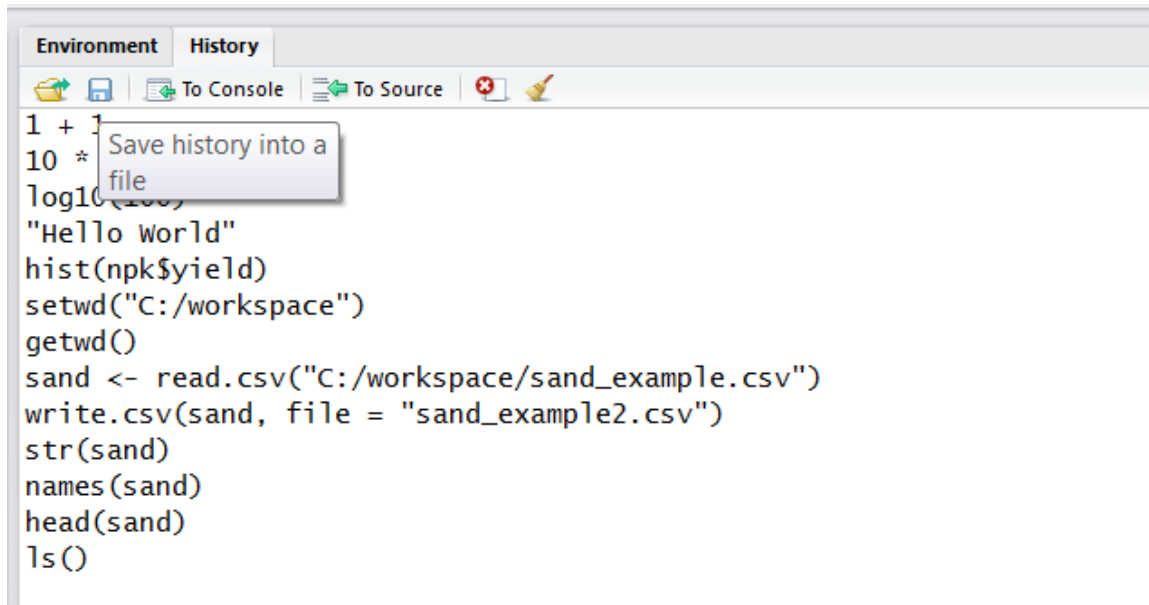


The R command for saving your workspace is:

`save.image(file="name of your workspace.RData")`

c) R history (.Rhistory)

An R history file is a copy of all your key strokes. You can think of it as brute force way of saving your work. It can be useful if you didn't document all your steps in an R script file. Like an R file, an Rhistory file is simply a text file that lists all of the commands that you've executed. It does not keep a record of the results. To load or save your R history from the History Tab click the **Open File** or **Save** button. If you load an Rhistory file, your previous commands will again become available with the up-arrow and down-arrow keys.



The screenshot shows the RStudio Environment pane with the 'History' tab selected. A tooltip 'Save history into a file' is visible over the first few lines of code. The code in the history includes:

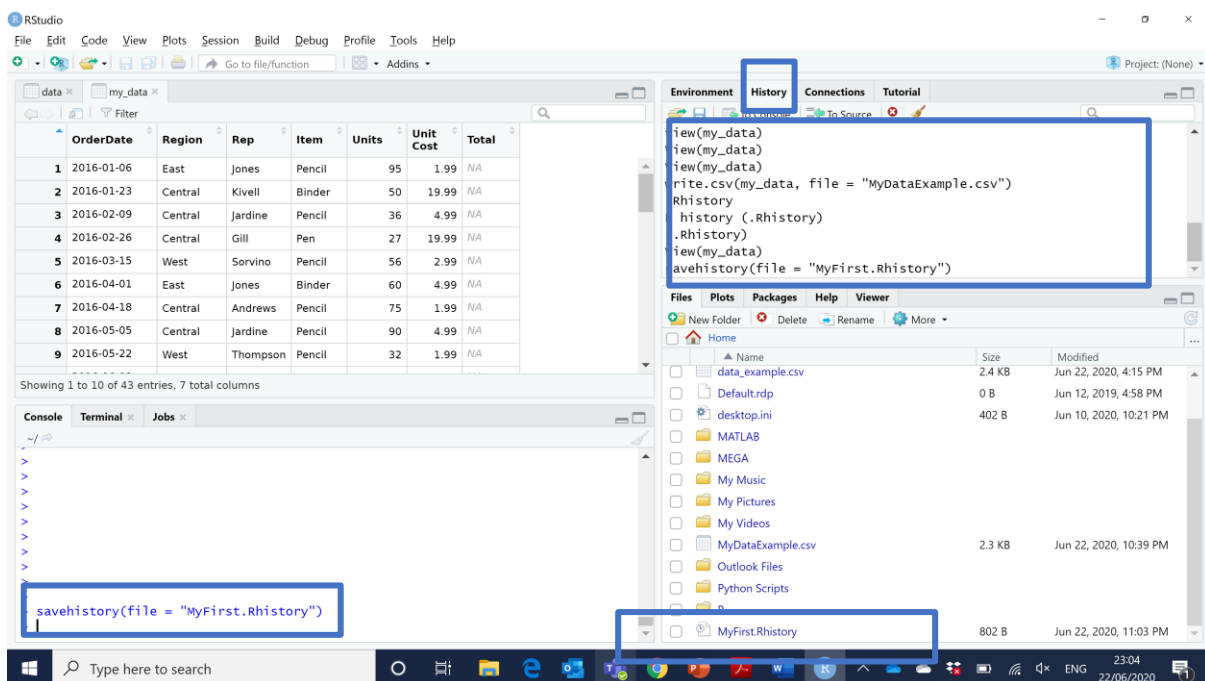
```
1 + 1
10 *
log10(100)
"Hello World"
hist(npk$yield)
setwd("C:/workspace")
getwd()
sand <- read.csv("C:/workspace/sand_example.csv")
write.csv(sand, file = "sand_example2.csv")
str(sand)
names(sand)
head(sand)
ls()
```

You can also use the command line to load or save your history.

```
savehistory(file = "MyFirst.Rhistory")
```

```
loadhistory(file = "MyFirst.Rhistory")
```

```
history(max.show=Inf) #displays all previous commands
```



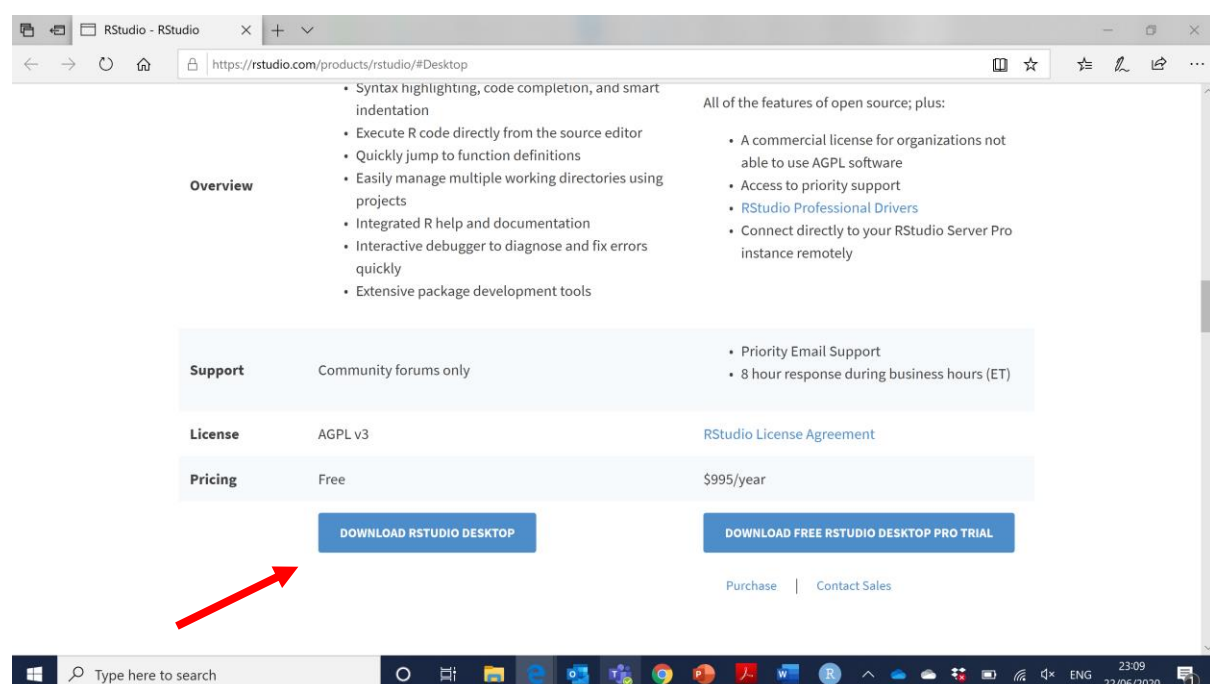
Some useful tips for using RStudio (Source: <http://ncss-tech.github.io>)

- R is command-line driven. It requires you to type or copy-and-paste commands after a command prompt (>) that appears when you open R. After typing a command in the R console and pressing **Enter** on your keyboard, the command will run. If your command is not complete, R issues a continuation prompt (signified by a plus sign: +). Alternatively, you can write a script in the script window, and select a command, and click the **Run** button.
- R is case sensitive. Make sure your spelling and capitalization are correct.
- Commands in R are also called functions. The basic format of a function in R is: `object <- function.name(argument_1 = data, argument_2 = TRUE)`.
- The up arrow (^) on your keyboard can be used to bring up previous commands that you've typed in the R console.
- The \$ symbol is used to select a particular column within the table (e.g., `table$column`).
- Any text that you do not want R to act on (such as comments, notes, or instructions) needs to be preceded by the # symbol (a.k.a. hash-tag, comment, pound, or number symbol). R ignores the remainder of the script line following #. For example: `plot(x, y) # This text will not affect the plot function because of the comment ```

7. Setting up R/RStudio on home PC/Laptop

To download/install the R, RStudio, follow the instruction on the Website:

<https://www.rstudio.com/products/rstudio/#Desktop>



8. Getting Started with R (Source: <https://www.r-project.org/>)

R is a free software environment for statistical computing and graphics. It compiles and runs on a wide variety of UNIX platforms, Windows and MacOS. To **download R**, please choose your preferred CRAN mirror.

If you have questions about R like how to download and install the software, or what the license terms are, please read our answers to frequently asked questions before you send an email.

Updates

- **R version 3.6.1 (Action of the Toes)** has been released on 2019-07-05.
- useR! 2020 will take place in St. Louis, Missouri, USA.
- **R version 3.5.3 (Great Truth)** has been released on 2019-03-11.
- The R Foundation Conference Committee has released a call for proposals to host useR! 2020 in North America.
- You can now support the R Foundation with a renewable subscription as a supporting member
- The R Foundation has been awarded the Personality/Organization of the year 2018 award by the professional association of German market and social researchers.

Recommended Reading

- a. Quick-R
- b. R Cookbook
- c. Reproducible Research with R & RStudio (not free)
- d. R Programming for Data Science, Roger D. Peng, 2019-09-18, [Online Available], <https://bookdown.org/rdpeng/rprogdatascience/getting-data-in-and-out-of-r.html>
- e. Efficient R programming, Colin Gillespie, Robin Lovelace, 2016-11-30, [Available Online] <https://bookdown.org/csgillespie/efficientR/>
- f. Hands-On Programming with R, *Garrett Golemund*, [Available Online], <https://rstudio-education.github.io/hopr/index.html>
- g. R for Data Science, *Garrett Golemund*, *Hadley Wickham*, [Available Online], <https://r4ds.had.co.nz/introduction.html#rstudio>

References

- Stephen Roecker, Skye Wills, Katey Yoast and Tom D’Avello. 2019. Chapter 1 Introduction to R and RStudio, [Available Online]: http://ncss-tech.github.io/stats_for_soil_survey/chapters/1_introduction/1_introduction.html#331_importing_data
- Ihaka, R., and R. Gentleman. 1996. R: A language for data analysis and graphics. *Journal of Computational and Graphical Statistics* 5(3):399–414, [Available Online] <https://www.stat.auckland.ac.nz/~ihaka/downloads/R-paper.pdf>